

GitHub 협업 규칙 쉽게 이해하기

비전공자와 신규 팀원을 위한 Lemon Healthcare PR1 협업 가이드

기준 문서: pr1/docs/05-github-guidelines.md, .github 설정 파일, pr1/.pre-commit-config.yaml
생성일: 2026-05-09 | 대상: 기획자, 발표 담당자, 데이터 담당자, 개발 입문자

이 문서는 GitHub를 처음 쓰는 팀원이 '무엇을 언제 해야 하는지' 이해하도록 현재 저장소의 규칙을 쉬운 말로 다시 정리한 PDF입니다. 세부 명령어보다 협업 흐름, 역할, 안전장치를 먼저 설명합니다.

1. 이 규칙의 목적

GitHub 규칙은 개발자를 통제하기 위한 문서가 아니라, 여러 사람이 동시에 작업해도 파일이 충돌하지 않고 발표 가능한 안정본을 지키기 위한 팀 운영 규칙입니다.

- **기록:** 누가, 언제, 어떤 이유로 바꿨는지 남깁니다.
- **검토:** 중요한 변경은 혼자 바로 합치지 않고 한 번 더 확인합니다.
- **자동검사:** 사람이 놓치는 포맷, 테스트, 링크 문제를 CI가 대신 확인합니다.
- **보안:** API 키, 비밀번호, 사용자 데이터가 GitHub에 올라가지 않게 막습니다.

비유하면, GitHub는 팀의 공동 작업장이고, 브랜치는 개인 작업 책상, PR은 제출 전 검토 요청서, CI는 자동 품질검사입니다.

2. 한 장으로 보는 작업 흐름

순서	팀원이 하는 일	쉽게 말하면
1	Issue 생성 또는 선택	무슨 일을 할지 먼저 표로 접수합니다.
2	develop에서 작업 브랜치 생성	공동 원본을 복사해 개인 작업 책상을 만듭니다.
3	작업 후 commit	작업 내용을 작은 단위로 저장하고 설명을 붙입니다.
4	push 후 Pull Request 생성	팀에 '이 변경을 합쳐도 되는지 봐주세요'라고 요청합니다.
5	CI 자동검사	테스트, 포맷, 문서 링크를 자동으로 확인합니다.
6	리뷰어 확인	최소 1명이 변경 내용을 보고 승인하거나 수정을 요청합니다.
7	Squash and Merge	여러 작업 기록을 깔끔하게 하나로 묶어 develop에 합칩니다.

핵심 원칙: main은 발표용 안정본, develop은 통합 개발본, feature/bugfix/docs 브랜치는 개인 또는 소규모 작업 공간입니다.

3. 꼭 알아야 할 용어

용어	비전공자용 설명
Repository	프로젝트 파일과 작업 기록이 모이는 온라인 보관함입니다.
Branch	원본을 건드리지 않고 따로 실험하거나 작업하는 복사본입니다.
main	발표나 시연에 사용할 안정 버전입니다. 직접 수정하지 않습니다.
develop	팀원이 완성한 작업을 먼저 모아보는 통합 버전입니다.
Commit	작업 저장 기록입니다. 변경 이유를 메시지로 남깁니다.
Pull Request	내 작업을 팀 공용 브랜치에 합치기 전 검토 요청입니다.
CI	GitHub가 자동으로 실행하는 검사입니다. 빨간 X면 고쳐야 합니다.
CODEOWNERS	어떤 폴더를 누가 검토해야 하는지 정해두는 파일입니다.
Secret	API 키, 비밀번호처럼 절대 코드에 올리면 안 되는 값입니다.

4. 브랜치 규칙

브랜치	역할	팀원이 기억할 점
main	발표·시연용 안정본	직접 작업하지 않고, 검토가 끝난 내용만 들어갑니다.
develop	개발 통합본	기능 브랜치의 PR이 모이는 기본 목적지입니다.
feature/*	새 기능 작업	예: feature/algo-v1-step-score
bugfix/*	버그 수정	예: bugfix/123-bmr-calculation-error
hotfix/*	긴급 수정	main에 문제가 생겼을 때만 사용합니다.
docs/*	문서 작업	기획서, README, 가이드 문서 수정에 사용합니다.

브랜치 이름은 소문자와 하이픈을 사용합니다. 짧고 명확해야 하며, 이슈 번호가 있으면 포함합니다.

예시: feature/ocr-pipeline-mvp, docs/06-tech-stack-update, bugfix/123-healthkit-permission

5. 커밋 메시지 규칙

커밋 메시지는 작업 기록의 제목입니다. 나중에 문제를 찾거나 발표 자료를 만들 때 무엇이 바뀌었는지 빠르게 추적하게 해줍니다.

<type>(<scope>): <subject>

type	의미	예시
feat	새 기능	feat(algo): add v1 step score calculation
fix	버그 수정	fix(ocr): handle empty supplement label
docs	문서 수정	docs(readme): update setup guide
test	테스트 추가	test(algo): add BMI edge cases
ci	자동검사 설정	ci: add docs link check workflow

나쁜 예: update, fix bug, WIP, 수정함. 이유는 나중에 봤을 때 무엇을 왜 바꿨는지 알 수 없기 때문입니다.

6. Issue와 PR 규칙

Issue는 할 일을 접수하는 카드이고, PR은 작업물을 팀에 제출하는 검토 요청서입니다.

구분	언제 사용하나	현재 템플릿
Feature Request	새 기능을 제안하거나 시작할 때	.github/ISSUE_TEMPLATE/feature_request.md
Bug Report	오류가 발생했을 때	.github/ISSUE_TEMPLATE/bug_report.md
Question	설계 의도나 사용법을 물을 때	.github/ISSUE_TEMPLATE/question.md
Pull Request	작업을 develop/main에 합치기 전	.github/PULL_REQUEST_TEMPLATE.md

- PR 제목은 커밋 메시지처럼 작성합니다. 예: feat(mobile): add onboarding agreement screen
- PR 설명에는 변경 내용, 테스트 방법, 관련 이슈, 민감정보 여부를 적습니다.
- 리뷰는 사람을 지적하지 않고 코드와 문서를 기준으로 말합니다.
- develop 병합은 최소 1명 리뷰, main 병합은 최소 2명 리뷰가 원칙입니다.

7. 자동검사 CI는 무엇을 보나

CI는 GitHub Actions가 PR이나 push 시점에 자동으로 실행하는 품질검사입니다. 사람이 매번 손으로 확인하기 어려운 문제를 먼저 잡습니다.

워크플로	언제 실행되나	검사 내용
Backend CI	pr1/backend 변경 시	Python 포맷, 린트, 타입 체크, pytest, 커버리지
Mobile CI	pr1/mobile 변경 시	Flutter doctor, pub get, dart format, flutter analyze, flutter test, debug APK 빌드
Docs CI	pr1 문서 변경 시	Markdown 문법 검사, 깨진 링크 확인

CI가 빨간 X로 실패하면 PR을 합치기 전에 원인을 고쳐야 합니다. 실패 화면에서 Details를 눌러 어떤 단계가 실패했는지 확인합니다.

8. 보안 규칙

API 키와 비밀번호는 한 번 GitHub에 올라가면 기록에 남을 수 있습니다. 그래서 저장소 규칙은 민감정보를 코드에 넣지 않는 것을 매우 중요하게 봅니다.

절대 올리면 안 되는 것	대신 해야 할 일
Google Cloud, Anthropic, OpenAI API 키	로컬 .env 또는 GitHub Actions Secrets에 보관
데이터베이스 비밀번호, JWT secret	.env.example에는 이름만 적고 실제 값은 비워둠
사용자 개인정보, 건강정보 샘플	익명 샘플 또는 가짜 데이터만 사용
Apple/Google 인증서, Service Account JSON	팀 보안 저장소나 GitHub Secrets 사용

실수로 키를 올렸다면 삭제만 하면 끝이 아닙니다. 즉시 해당 키를 폐기하고 새 키를 발급해야 합니다.

9. 현재 저장소에 있는 실제 설정

파일	역할
pr1/docs/05-github-guidelines.md	GitHub 협업 규칙 원문
.github/PULL_REQUEST_TEMPLATE.md	PR 작성 시 자동으로 채워지는 체크리스트
.github/ISSUE_TEMPLATE/*.md	버그, 기능 제안, 질문용 이슈 양식
.github/CODEOWNERS	현재 모든 영역 리뷰어를 @HorangEe02로 지정
.github/workflows/ci-backend.yml	백엔드 자동검사
.github/workflows/ci-mobile.yml	모바일 자동검사
.github/workflows/ci-docs.yml	문서 자동검사
pr1/.pre-commit-config.yaml	커밋 전 로컬 자동검사 설정

10. 관리자 확인이 필요한 적용상 주의

로컬에서 확인한 Git 루트는 /Users/yeong/99_me/00_github 이고, 현재 작업 폴더는 그 하위의 03_lemon_healthcare 입니다. GitHub Actions는 일반적으로 저장소 루트의 .github/workflows 아래에 있는 워크플로를 인식합니다.

- 03_lemon_healthcare가 GitHub에서 독립 저장소라면 현재 .github/workflows 구조가 맞습니다.
- GitHub 저장소 루트가 /Users/yeong/99_me/00_github와 동일한 구조라면, 03_lemon_healthcare/.github/workflows는 Actions에서 인식되지 않을 수 있습니다.
- pr1/.pre-commit-config.yaml도 팀 전체 기본 혹은 쓰려면 실행 위치와 설치 절차를 명확히 해야 합니다.
- CODEOWNERS의 @HorangEe02는 현재 placeholder 성격이므로 실제 팀 역할이 정해지면 영역별 담당자로 바꾸는 것이 좋습니다.

11. 비전공자용 실전 체크리스트

상황	확인할 것
작업 시작 전	이슈가 있는지 확인하고, 없다면 Feature/Bug/Question 이슈를 만듭니다.
브랜치 만들 때	develop에서 시작하고 이름을 feature/..., bugfix/..., docs/... 형식으로 만듭니다.
커밋할 때	feat, fix, docs, test 중 맞는 type을 고르고 구체적인 제목을 씁니다.
PR 만들 때	템플릿의 요약, 테스트, 체크리스트, 관련 이슈를 채웁니다.
리뷰할 때	문제만 말하지 말고 왜 바꾸면 좋은지와 대안을 같이 적습니다.
CI 실패 시	빨간 X의 Details를 열고 실패 단계와 로그를 담당자에게 공유합니다.
민감정보 발견 시	즉시 공유를 중단하고 키 폐기/재발급을 먼저 진행합니다.

가장 중요한 한 줄: main은 건드리지 말고, develop에서 새 브랜치를 만들어 작업한 뒤 PR로 검토받고 합칩니다.

12. 공식 참고 문서

아래 URL은 GitHub 기능과 커밋 메시지 규칙을 확인할 때 참고한 공식 문서입니다.

주제	공식 URL
Pull Request	https://docs.github.com/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-pull-requests
Branch	https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-branches
Protected Branch	https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches
CODEOWNERS	https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-code-owners
GitHub Actions	https://docs.github.com/en/actions/learn-github-actions/understanding-github-actions
Workflow syntax	https://docs.github.com/actions/using-workflows/workflow-syntax-for-github-actions
Actions Secrets	https://docs.github.com/actions/reference/encrypted-secrets
Issue/PR templates	https://docs.github.com/en/communities/using-templates-to-encourage-useful-issues-and-pull-requests/about-issue-and-pull-request-templates
Conventional Commits	https://www.conventionalcommits.org/en/about